Bagging Regressor

Contents

1. Overview 2
2. The core idea 2
3. Seeing it on a curve 4
4. BaggingRegressor on the Boston housing data 6
    - 4.1 Three single regressors 6
    - 4.2 A bagging regressor with default settings 7
    - 4.3 Tuning with GridSearchCV 7
    - 4.4 The out-of-bag score 8
5. Summary 10
6. Sources 10
7. Key terms 10

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate), 10 (Tune). See the Course map.



- **Builds on:** Grid and random search; Bagging.
- **Used here, taught in full later:** OOB score.
- **Compare with:** Cross-validation; Voting ensembles; Random forest; Boosting; Bayesian optimisation; Optuna; Keras Tuner.

1. Overview

Key point: A bagging regressor works exactly like a bagging classifier, except that aggregation takes the mean of the base models' numbers instead of a **majority vote** (G-1146).

The **bagging regressor** (G-252) applies **bagging** (G-251; training many models on random samples of the data and combining their answers, see the core idea) to regression, where the answer is a number. Everything carries over from the bagging classifier: the four sampling types and every hyperparameter. This Note covers:

- the one difference, in the **aggregation** (G-183) step;
- a demo with curves: one tree against a bagged ensemble;
- `BaggingRegressor` on the Boston housing data, tuned with `GridSearchCV`.

The Notebook (`ML-101-bagging-regressor.ipynb`) runs every experiment. The Dash app `app.py` lets us change the **base model** (G-260) and the bagging settings and redraws both curves.

2. The core idea

Key point: Bootstrapping is unchanged; aggregation averages the predictions.

In the **bootstrapping** step, each **base model** (G-260), one of the models being combined, is trained on its own **bootstrap sample** (G-319): observations drawn at random with replacement, as in the classifier. The other sampling types carry over too, so **pasting** (G-1463), random subspaces and random patches are all available.

In the **aggregation** (G-183) step, each base model returns a number, since this is regression. The **bagging regressor** (G-252) returns their **mean**. Taking the mean is the only change.

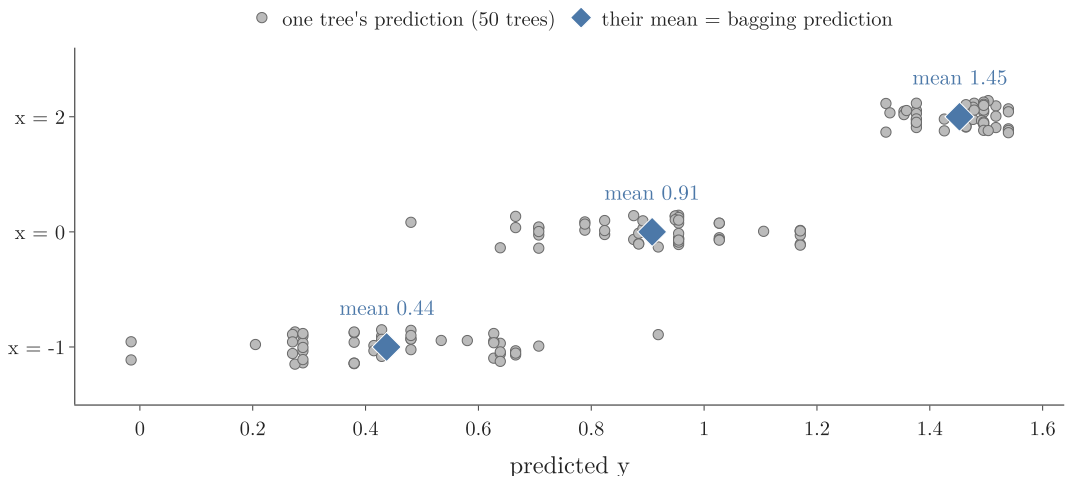


Figure 1: Aggregation in a bagging regressor: at three inputs x, each of the 50 bagged trees of section 3 returns a number (grey dots); the bagging prediction is their mean (blue diamond)

Figure 1 shows the aggregation step on the 50 trees of section 3, trained on a small dataset with one input x and a number as the target (section 3 describes it). Watch how far apart the single trees land at x = -1 (from about 0.0 to 0.9), while the mean sits in the middle of the cloud: no tree is trusted on its own.

The aggregation step, on the numbers of Figure 1:

1. **In words:** ask every base model for its number, add the numbers up, and divide by how many models there are.
2. **Formula:** with M base models $f_1, \ldots, f_M$, the bagging prediction at an input x is

$$\hat{y}(x) = \frac{1}{M} \sum_{m=1}^{M} f_m(x)$$

   where $f_m(x)$ is the number that base model m returns for x.

3. **Example:** at x = -1, the first three trees return 0.38, 0.67 and 0.29, whose mean is:

$$(0.38 + 0.67 + 0.29)/3 = 0.45$$

   Over all $M = 50$ trees the mean is **0.44**, the blue diamond of Figure 1, and exactly what `predict` returns.

A classifier replaces only this step: it counts the base models' classes and returns the **majority vote** (G-1146).

scikit-learn's `BaggingRegressor` (G-254) has exactly the same hyperparameters as `BaggingClassifier` (see what works in practice): `estimator`, `n_estimators`, `max_samples`, `max_features`, `bootstrap`, `bootstrap_features` and `oob_score`.

3. Seeing it on a curve

> **Key point:** One fully grown tree passes through every training point (test R^2 0.92); 50 bagged trees give a smooth curve that follows the pattern (0.95).

The demo data has one **feature** (input variable, one column of the data table). Its **target** (the output we predict) is two Gaussian bumps (a small one at 0 and a larger one at 2) plus a little noise, with 150 training **observations** (points, one row of the table each) and 100 test points. With a single feature there are no features to sample, so the app only offers observation settings: the **base model** (G-260) (decision tree, SVR or KNN), `n_estimators`, `max_samples` and `bootstrap`.

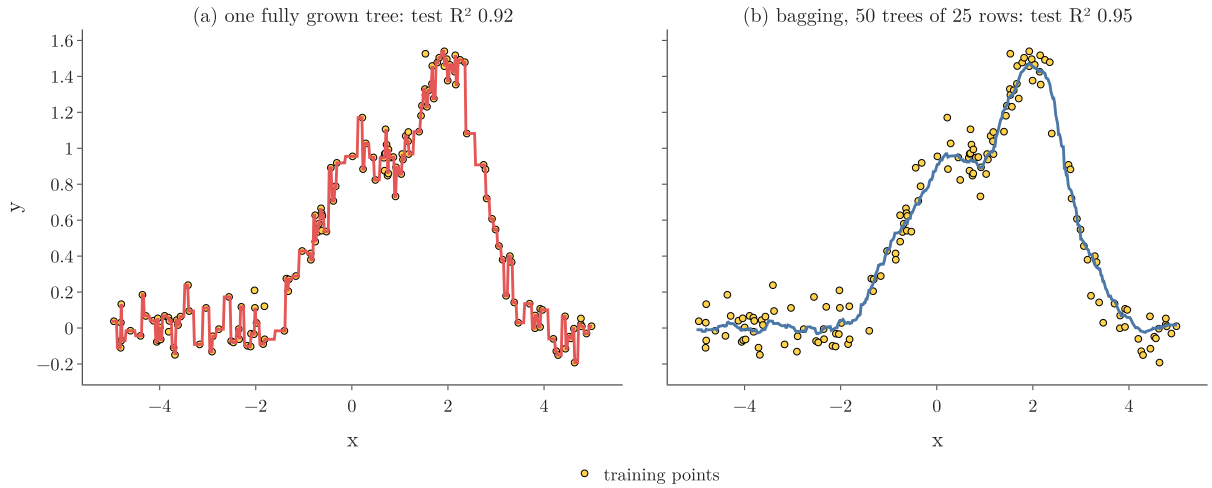


Figure 2: (a) One fully grown regression tree; (b) a bagging regressor of 50 trees, each trained on 25 observations drawn with replacement

Figure 2 compares a single decision tree with bagging (50 trees, 25 observations each, with replacement):

- **(a) The single regression tree** (G-1654) tries to pass through every training point, outliers included. Such a model is **low bias, high variance** (G-1132): it can follow any pattern, but its prediction swings with the particular training points. Fitting the noise in this way is **overfitting** (G-1429). Test R^2: **0.92** (the **R^2 score** (G-1717): the share of the target's variation the model explains; 1 is perfect).
- **(b) The bagging regressor** gives a smooth curve. The curve does not touch every point; it follows the underlying pattern. Test R^2: **0.95**.

Why the mean is smoother, step by step:

1. A regression tree predicts one constant value per leaf, so its prediction is a **step function** (G-1889): flat inside each leaf, with a jump at every split.
2. Each bagged tree saw a different **bootstrap sample** (G-319), so it puts its jumps at different values of x (Figure 3, grey trees).

3. The mean of 50 step functions with jumps in different places has many small steps instead of a few large ones, which looks like a smooth curve.

In the terms of the bias-variance trade-off (G-288), each tree keeps its low bias, and averaging removes much of its **variance** (G-2074), the swing of its prediction from one training sample to another (Breiman, 1996, section 1).

Figure 3 builds the **bagging regressor** (G-252) of Figure 2b one tree at a time. Each new tree (green) sees only its 25 drawn points, so its steps fall in different places from the trees before it (grey). Watch the blue mean: wherever the trees disagree, their steps cancel out, and the mean turns from one tree's jagged staircase (test R^2 0.77, below the 0.92 of Figure 2a because this tree saw only 25 of the 150 points) into a smooth curve (0.95 with 50 trees).

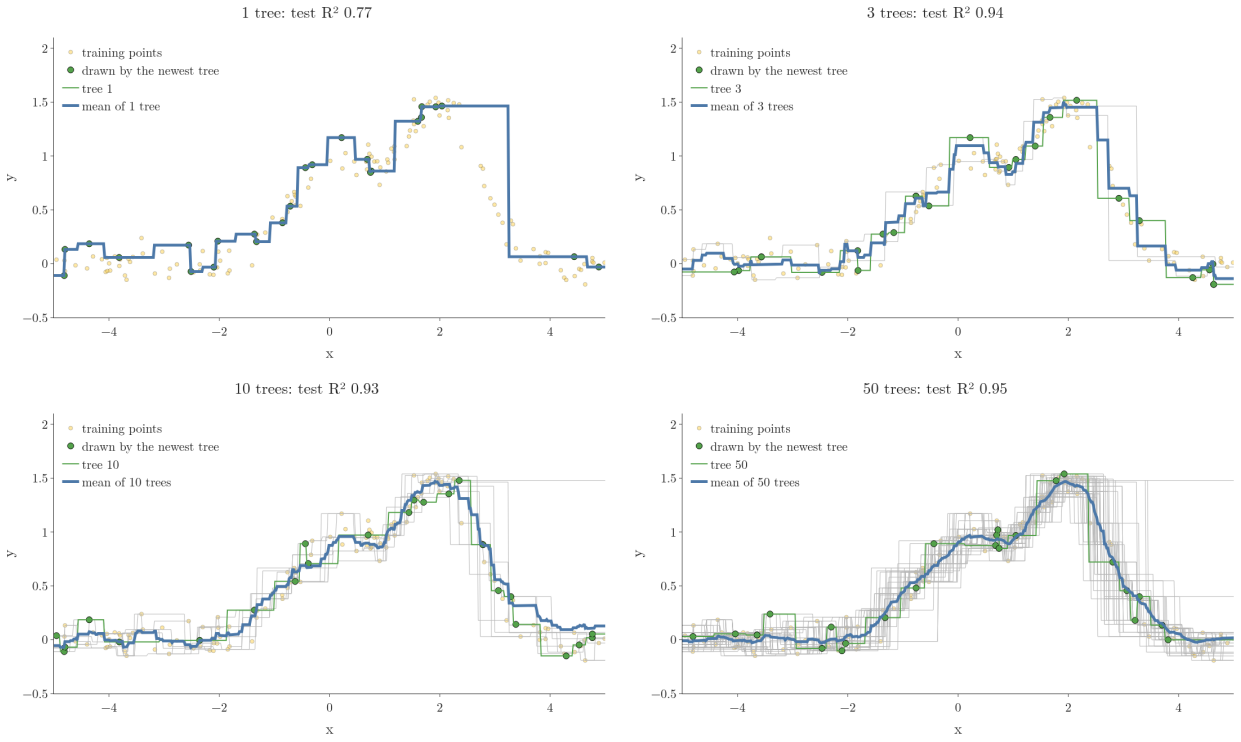


Figure 3: The bagging regressor built one tree at a time: every tree so far (grey), the newest tree and its 25 drawn points (green), and the mean of the trees, the bagging prediction (blue)

Changing the settings makes little difference on this easy data: 100 trees score 0.952, and **pasting** (G-1463) (75 observations, `bootstrap=False`) scores 0.947.

> **Extra:** Bagging helps unstable models, whose fit swings with small changes in the data, such as trees; it can slightly hurt stable ones, such as nearest-neighbour methods (Breiman, 1996, sections 1 and 6.3; other base models). The bumps data agrees. With the 25-observation samples of Figure 2, bagged KNN (0.832) and bagged SVR (0.871) both do worse than a single KNN (0.954) or SVR (0.894), because each copy sees only 25 points. With full-size samples of 150, bagging brings no gain: KNN 0.956 against 0.954,

SVR 0.894 against 0.894 (app `fit` function, `max_samples=150`).

4. BaggingRegressor on the Boston housing data

Key point: Out of the box, bagged trees (mean test R^2 0.84) beat linear regression (0.71), a single tree (0.72) and KNN (0.50).

4.1 Three single regressors

Key point: Averaged over 100 random splits: linear regression 0.71, decision tree 0.72, KNN 0.50.

The Boston housing data (see the data and a first tree) has 506 districts (observations), 13 features and the median house price as the target. We split it into 404 training observations and 102 test observations, and train three single regressors.

A test set of 102 observations is small, so one split gives a noisy score: on our first split, for example, the single tree scores only 0.38. So the Notebook repeats the split 100 times, with a different random shuffle each time, and every number in this section is the mean over those 100 splits:

Model	Linear regression	Decision tree	KNN
Mean test R^2	0.708	0.716	0.504

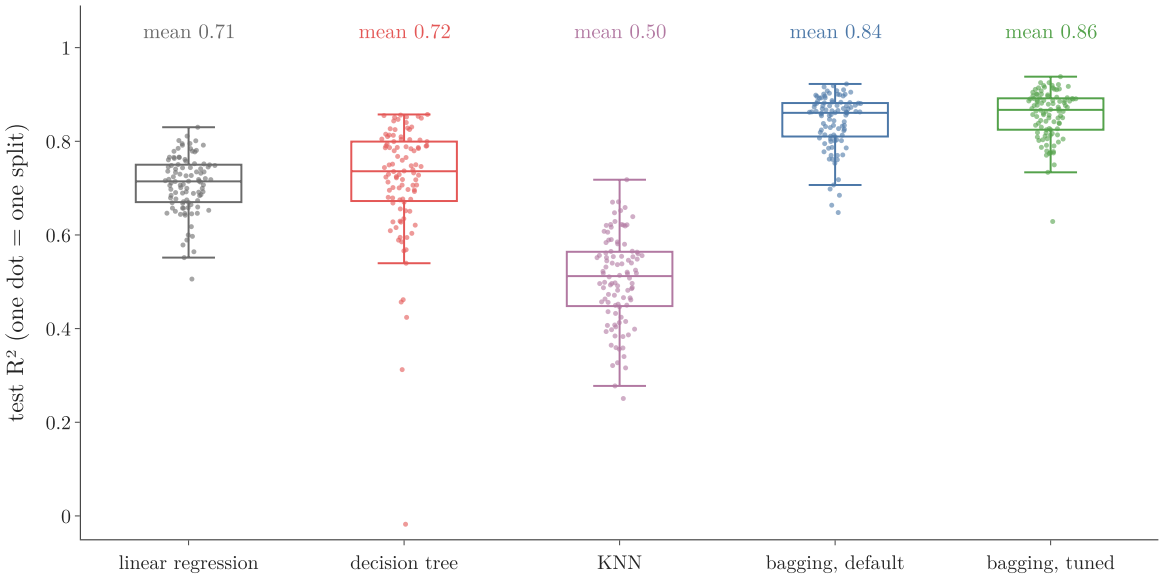


Figure 4: Test R^2 of five regressors on the same 100 random splits of the Boston data: each dot is one split, the box holds the middle half, and the number on top is the mean

Figure 4 shows all 100 scores per model, including the two **bagging regressors** (G-252) of sections 4.2 and 4.3. Watch the single tree's spread: from about 0.0 to 0.86 depending on the split, while both bagging boxes sit higher and tighter.

4.2 A bagging regressor with default settings

> **Key point:** Ten bagged trees, nothing tuned: mean test R^2 0.84, above every single model.

> **Python:** A **bagging regressor** (G-252) with every setting at its default.

```
from sklearn.ensemble import BaggingRegressor

bag = BaggingRegressor(random_state=1)
bag.fit(X_train, y_train)
bag.score(X_train, y_train)     # 0.980 (first split)
bag.score(X_test, y_test)       # 0.828 (first split)
```

> The defaults: `estimator=None` (a decision tree), `n_estimators=10`, as many observations per model as the training set (`max_samples=None`), drawn with replacement, and every feature. `score` returns R^2 for a regressor.

Without tuning anything, the bagging regressor scores a mean test R^2 of **0.841** over the 100 splits (Figure 4, blue), well above all three single models (0.708, 0.716 and 0.504). The ten trees are each as unstable as the single tree, but their average is far steadier. On the first split, the gap between training (0.98) and test (0.83) shows some **overfitting** (G-1429) remains.

4.3 Tuning with GridSearchCV

> **Key point:** A grid search over 144 combinations picks 50 bagged trees on full **bootstrap samples** (G-319); over the 100 splits these settings score 0.857, above the default's 0.841.

Instead of trying **pasting** (G-1463), **random subspaces** (G-1618) and **random patches** (G-1614) by hand, we let `GridSearchCV`, a **grid search** (G-872), try them all (as in experiments: try every k):

> **Python:** Searching over the **base model** (G-260) and the sampling settings.

```
params = {
    "estimator": [None, LinearRegression(),
                  KNeighborsRegressor()],
    "n_estimators": [20, 50, 100],
    "max_samples": [0.5, 1.0],
    "max_features": [0.5, 1.0],
    "bootstrap": [True, False],
    "bootstrap_features": [True, False]}
grid = GridSearchCV(BaggingRegressor(random_state=1),
```

```
                    param_grid=params, cv=3,
                    n_jobs=-1, verbose=1)
grid.fit(X_train, y_train)
grid.best_params_
```

The base model itself is a hyperparameter here: `None` stands for the default decision tree. With `bootstrap` and `bootstrap_features` both tried as `True` and `False`, the search covers bagging, pasting, random subspaces and random patches.

The grid holds 144 combinations:

$$3 \times 3 \times 2 \times 2 \times 2 \times 2 = 144$$

Each has 3-fold **cross-validation** (G-510; the training data is cut into 3 parts, and each part is scored once by a model trained on the other two): 432 fits, under a minute with `n_jobs=-1`. The best settings:

- base model: **decision tree**;
- `bootstrap=True`: **bagging**, not pasting;
- `bootstrap_features=False`, `max_features=1.0`: **no feature sampling**;
- `max_samples=1.0`: every model gets as many observations as the training set;
- `n_estimators=50`.

Their cross-validation R^2 is **0.871**. On the same 100 splits as above, these settings score a mean test R^2 of **0.857** (Figure 4, green), close to the cross-validation estimate and above the default **bagging regressor** (G-252) (0.841); they beat the default on 79 of the 100 splits.

Model	Linear regression	Decision tree	KNN	Bagging, default	Bagging, tuned
Mean test R^2 (100 splits)	0.708	0.716	0.504	0.841	**0.857**

Extra: One split can mislead. On the first split alone, the tuned model scores 0.806 and the default 0.828, the opposite order. Over the 100 splits, the tuned settings score anywhere from 0.63 to 0.94, so one 102-observation test score can land far from the average. The search picks settings that do best on average, which is what the mean over many splits measures. Expecting one test score to match `best_score_` exactly is a common mistake.

4.4 The out-of-bag score

Key point: With `oob_score=True`, the regressor reports R^2 on the out-of-bag observations: 0.870 here.

The **out-of-bag score** (G-1411) works for regression too (see the out-of-bag score). For a regressor, `oob_score_` is the R^2 on the observations each model never saw. With 50 trees on the first split, the OOB R^2 is **0.870**, close to the cross-validation score.

5. Summary

	Bagging classifier	Bagging regressor
Base models	classifiers	regressors
Aggregation	majority vote	mean
scikit-learn	`BaggingClassifier`	`BaggingRegressor`
`score` and `oob_score_`	accuracy	R^2
Hyperparameters	`estimator`, `n_estimators`, `max_samples`, `max_features`, `bootstrap`, `bootstrap_features`, `oob_score`	the same

- Only **aggregation** (G-183) changes for regression: the mean replaces the vote, because each base model returns a number, so everything learned for the classifier carries over.
- On the bumps data, bagging smooths one **overfitting** (G-1429) tree into a curve that follows the pattern: test R^2 0.92 to 0.95, because each tree puts its jumps in different places, and the mean of many such **step functions** (G-1889) has many small steps instead of a few large ones.
- On Boston (mean over 100 splits), default bagged trees score 0.84 against 0.72 for the best single model, because the average of ten unstable trees is far steadier than one; a **grid search** (G-872) picks plain bagging of 50 trees, which scores 0.86. We average over 100 splits because one 102-observation test score can land far from the average.
- Bagging helps unstable models such as trees; for stable KNN and SVR it brings no gain (full-size samples) or a loss (25-observation samples), because their fit barely swings with the data, so there is little **variance** (G-2074) to average away.
- So a bagging regressor is the bagging classifier with a mean in place of a vote, and it cuts the variance of an overfitting **regression tree** (G-1654).

6. Sources

Built from

- CampusX, "Bagging Ensemble | Part 3 | Bagging Regressor", YouTube, https://www.youtube.com/watch?v=HYVzrETXbkE

Other references

- Breiman, L. (1996). "Bagging Predictors". *Machine Learning* 24(2), 123–140, sections 1 and 6.3.

7. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Bagging regressor (G-252)	A bagging ensemble of regressors that predicts the mean of their predictions.
BaggingRegressor (G-254)	scikit-learn class for bagging, pasting, random subspaces and random patches in regression.
Majority vote (G-1146)	Predicting the class that most voters choose: the k neighbours in KNN, or the base models of an ensemble.
Bagging (bootstrap aggregation) (G-251)	Training many models on different random samples of the data and averaging them, so the result depends less on the particular sample (lower variance).
Aggregation (in an ensemble) (G-183)	Combining the base models' predictions into one: mode for classes, mean for numbers.
Base model (G-260)	One of the models inside an ensemble; the ensemble combines the base models' answers into its prediction.
Bootstrap sample (G-319)	A sample of the same size as the data, drawn with replacement so some rows repeat and others are left out; each tree of a random forest gets its own, so the trees differ.
Pasting (G-1463)	Training several models, each on its own random sample of rows drawn without replacement (no row twice), and combining their predictions; it is bagging with this sample instead of a bootstrap sample.
Regression tree (G-1654)	A decision tree whose leaves predict numbers: the mean output of their training rows.
Low bias, high variance algorithm (G-1132)	An algorithm that fits its training data very well but changes a lot with the data, such as a fully grown tree; it overfits.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.
R^2 score (coefficient of determination) (G-1717)	1 minus the model's squared error divided by the squared error of always predicting the mean: 1 is perfect, 0 is no better than the average.
Step function (G-1889)	A function that outputs 1 for positive inputs and 0 otherwise; the classic perceptron uses it to turn its score into a class.
Bias-variance trade-off (G-288)	Lowering bias by adding complexity tends to raise variance, and the reverse.

Term	Meaning
Variance (of data) (G-2074)	The average squared distance of the values from their mean; the square of the standard deviation; divide by n for a population (NumPy default) or $n-1$ for a sample (pandas default).
Random subspaces (G-1618)	Training several models, each on all rows but a random subset of columns, and combining their predictions (a kind of bagging).
Random patches (G-1614)	Training several models, each on a random subset of rows and a random subset of columns, and combining their predictions (a kind of bagging).
Grid search (G-872)	Training a model for every combination of listed settings and keeping the best by cross-validation.
Cross-validation (G-510)	Testing a model by training and testing it several times on different parts of the training data.
Out-of-bag (OOB) evaluation, OOB score (G-1411)	Testing a bagging model by predicting each training row with only the base models that never saw it; the OOB score is the accuracy (or R^2) of those predictions.
Observation (G-1374)	One record of a dataset: one row of the data table.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.